

chatops-gym: Validated Task Curricula for RL Training of Software Agents on a Real Chat Application

Marcos Lahoz
marcoslahozy14@gmail.com

July 2026

Abstract

Reinforcement learning from verifiable rewards (RLVR) trains agents on tasks whose outcomes a deterministic program can grade. At curriculum scale, the binding constraint is not writing tasks but *trusting* them: a grader with a false-positive path poisons the reward signal, and a task whose reference solution silently exceeds the agent’s action surface wastes it. We present CHATOPS-GYM, a curriculum *generator* for a real Slack-like application (Mattermost) in which every task instance derives its prompt, world state, binary grader, and reference (oracle) solution from the same sampled template parameters, and no instance ships without passing an automated validation harness: the oracle must succeed *through the agent’s own tools*, and the grader must reject a suite of *mutants* — systematic near-miss perturbations of the oracle — alongside standard null/random controls. Evaluating two models spanning $20\times$ in scale (Qwen2.5-1.5B and 32B) shows difficulty is a property of the (task, model) pair: the small model fails mid-curriculum with act-without-verifying pathologies (retry spirals, dropped steps, distractor capture), while the large model saturates all quantitative difficulty knobs and is only brought into the useful reward band (40% pass) by a *structural* escalation — cross-channel aggregation with no retrieval shortcut — where it exhibits the inverse pathology: exploration without commitment. We argue this saturate–diagnose–escalate loop, with each step machine-validated, is the core workflow of curriculum engineering for RL.

1 Introduction

Frontier models are increasingly trained to *act* in real software — email clients, CRMs, chat workspaces — via reinforcement learning on verifiable tasks: scenarios with a programmatic pass/fail grader whose binary outcome serves as reward [1, 2]. Production pipelines consume such tasks by the thousands, which moves the engineering bottleneck away from any individual task and toward three systemic properties of the curriculum: *consistency* (does the grader check what the prompt asks?), *solvability* (can the agent, with its given tools and permissions, actually succeed?), and *calibration* (do tasks sit in the difficulty range where a pass rate between roughly 10% and 60% yields learning signal, rather than at 0% or 100% where gradients vanish?).

Hand-curated benchmarks address these properties with human review, which does not scale to generated curricula. CHATOPS-GYM addresses them by construction and by automated adversarial validation. Its contributions:

1. A **procedural task generator** over a real, containerized chat application in which prompt, world overlays, grader parameters, and oracle solution are all functions of the same sampled parameters — eliminating task/grader drift by construction (§3).
2. A **validation harness** whose central component is *mutation testing of graders*: each family defines named near-miss perturbations of its oracle solution (wrong location, superseded value, negated answer, collateral spam, partial aggregate), all of which the grader must reject. The oracle itself must pass *through the agent’s tool layer*, upgrading the guarantee from “solvable via the API” to “solvable by the agent” (§4).
3. An **empirical calibration methodology** treating the structural difficulty taxonomy as a prior and pass rate as a per-model measurement, demonstrated across a $20\times$ model-scale gap with annotated failure-mode analysis, including a saturation event resolved by structural (not parametric) difficulty escalation (§6).

2 Environment

The environment is Mattermost, an open-source Slack-class workspace, deployed unmodified as a single Docker container. The choice is deliberate: curriculum engineering treats the environment as a commodity — the work is in the tasks. Two properties make it suitable. First, its REST API exposes the complete world state (channels, posts, threads, pins, reactions, membership), so graders verify *final state* deterministically, with no scraping or vision.

Level	Family	Structural definition
L1	<code>channel_setup</code>	One action; target entity named explicitly in the prompt.
L2	<code>channel_setup</code>	2–4 chained actions (create, configure, invite); no retrieval.
L3	<code>thread_reply</code>	Cross-channel retrieval among seeded distractors, then a threaded action; indirect references.
L4	<code>rollup_reply</code>	Cross-channel <i>aggregation</i> : the target number exists nowhere and must be computed from n component figures, each hidden among superseded drafts; single threaded reply.

Table 1: Difficulty levels as structural priors.

Second, its object model (channels, threads, mentions, pins) supports a task space rich enough to span four difficulty levels without leaving the application.

Each episode runs in a **fresh Mattermost team** seeded deterministically: a fixed 12-user roster, 8 channels, and n background messages generated from a world seed (deterministic RNG + seeded Faker), forming natural distractor material. Team-per-episode namespacing yields clean isolation, trivial snapshot scoping, and a direct concurrency story (N parallel rollouts = N teams). Entity IDs are not stable across resets; all references anchor on stable keys (channel names, unique marker texts), never IDs.

The agent acts through 13 tools wrapping the API (read/search/thread/post/react/pin/channel-management/finish), with a 20-call episode budget. Backends are pluggable (Ollama, OpenAI- and Anthropic-compatible endpoints, local HuggingFace); in this report inference ran on a Colab GPU behind a tunnel, with the environment and harness local.

3 Task Schema and Generation

A task instance is a self-contained JSON object: identity and difficulty metadata; a *setup* (world seed plus *overlays* — the task’s deltas, e.g. an unanswered question posted by a seeded user, each registering a named *marker* for the entity it creates); a parameterized grader reference; and a mandatory `oracle_solution`, the proof of solvability, expressed as tool calls.

Templates are pure functions $\text{seed} \mapsto \text{instance}$. Because the answer value, its hiding place, the near-miss decoys, the grader’s expected value, and the oracle’s actions are all derived from the same draw, consistency between what the prompt asks and what the grader checks holds *by construction* — the property hand-written curricula lose at scale. Templates additionally expose **continuous difficulty knobs** (number of near-miss distractors, haystack depth, components to aggregate) whose role appears in §6.

Four families currently span the taxonomy of Table 1. The taxonomy is structural — defined by properties of the task, not by observed pass rates — and is explicitly a *prior*; calibration is per-model (§5).

4 Graders and the Validation Harness

4.1 Grading model

Graders verify final state, as in τ -bench’s database comparison [1], hardened in two ways. **Snapshot-diff**: the environment snapshots the episode team at reset; graders diff the final state against it, so collateral damage (spam posts, stray channels, header vandalism) is detected generically without enumerating what the agent should not touch. **Typed answer matching**: expected values are typed (`money`, `text`, `number`) and matched with normalization ($\$48,500 \equiv 48500 \equiv 48.5k$), a negation guard (“definitely *not* \$48,500” fails), and ambiguity rejection (asserting the right value alongside a wrong one fails). Raw substring matching — common in quickly written graders — is vulnerable to all three. Every verdict carries a human-readable reason; debugging graders at scale without reasons is impractical.

4.2 Oracle through the agent’s tools

The harness executes each task’s oracle via the same tool registry, user account, and permissions as the evaluated agent. References inside oracle steps are declared as resolution specs (“the post in #q3-planning containing ⟨question fragment⟩”) and re-found through the tools; the marker registry is used only to cross-check that resolution found the intended entity, failing loudly otherwise. This closes a subtle gap: an oracle with raw API access would certify tasks requiring capabilities the agent’s tools do not expose.

Mutant	Grader defect it exposes
<code>channel_instead_of_thread</code>	Greps the channel for the answer; ignores threading.
<code>distractor_value / decoy_component_sum</code>	Accepts any seeded figure; no draft/final discrimination.
<code>negated_value</code>	Raw substring matching.
<code>hedged_answer</code>	No ambiguity rejection.
<code>correct_plus_spam</code>	Missing snapshot-diff collateral check.
<code>partial_sum / component_only</code>	Aggregate not actually verified.
<code>missing_header / wrong_member</code>	Chain steps not individually verified.
<code>noop / read_only</code>	Grader satisfied by the initial state.

Table 2: Mutant classes across families. Mutants derive from the same sampled parameters as the task, so they remain consistent with each instance automatically.

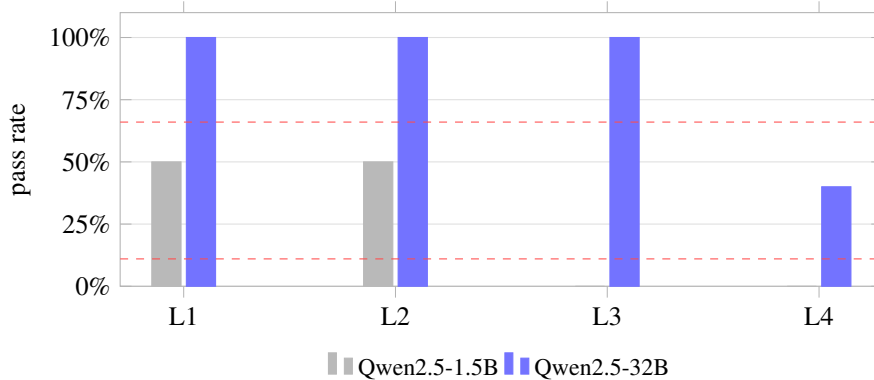


Figure 1: Pass rate by structural level for both models (dashed lines: the 10–60% useful-gradient band). The same curriculum is simultaneously too hard at L3 for the 1.5B and saturated through L3 for the 32B; only the structural L4 escalation places the 32B in band. (1.5B was not run on L4: it already fails at L3.)

4.3 Mutation testing

Null-agent and random-agent controls are standard but weak: with 13 tools and 20 calls, a random agent essentially never solves an L3 task, so passing that test says little. The false positives that matter in RL come from *plausible near-misses*, because a policy mid-training produces almost-correct behavior, not noise. Each family therefore defines **mutants**: named perturbations of the oracle derived from the instance’s own parameters (Table 2). The shipping contract per instance is: oracle $\rightarrow$ 1; every mutant $\rightarrow$ 0; null $\rightarrow$ 0; random (k seeds) $\rightarrow$ 0. The harness runs in CI on every commit; a family without registered mutants is rejected outright.

The harness demonstrated its value empirically during development, catching two real grader bugs before any model was evaluated: Mattermost attributes system join-messages to the acting user, which the collateral check initially counted as agent posts (caught by the oracle run); and the original substring matcher accepted negated answers (caught by the `negated_value` mutant). Later, a real 32B episode organically reproduced the `correct_plus_spam` mutant — correct threaded reply, plus the same answer duplicated as a channel post “to be safe” — confirming the mutant class models genuine policy behavior.

5 Per-Model Calibration

RLVR reward is binary, so a task’s contribution to learning depends on its pass rate under the *current* policy: saturated tasks ($\approx 100\%$) and impossible ones ($\approx 0\%$) contribute no gradient. We therefore treat the band of roughly 10–60% as the target zone, and — crucially — as a *per-model* property. The pipeline re-measures the full curve against any backend (a results file and one report command) and flags each family as TOO HARD, IN BAND, or SATURATED for that model, prescribing knob adjustments or level moves.

6 Experiments

Setup. Two instruction-tuned models from the same family, $20\times$ apart in scale: Qwen2.5-1.5B (local CPU, bf16, greedy) and Qwen2.5-32B (Q4, Ollama on an RTX PRO 6000, greedy), on curricula validated end-to-end by the harness of §4 (20 instances, 4 families, ~ 200 validation episodes, zero failing checks). Sample sizes are deliberately small ($n=2-5$ per cell) — this is a methodology demonstration; the pipeline re-runs at any n .

6.1 The small model: acting without verifying

The 1.5B passes half of L1/L2 and nothing at L3 (Fig. 1). Its four failures, annotated in the repository, form archetypes: *goal-completion blindness* (task complete at call 2; the model then invents extra objectives, misreads “already exists” as an obstacle, and creates -new/-new2 collateral channels); a *silently dropped step* behind a confident success summary — self-reports are not evidence, final-state grading is; *distractor capture plus a degenerate loop* (adopts a seeded near-miss figure, asks for confirmation instead of answering, then pastes the same message 18 times); and the organic *correct_plus_spam* reproduction noted above.

6.2 The large model: saturation, and a structural response

The 32B saturates the base curriculum 6/6 at near-optimal call counts (2–5). A -hard batch — 6 distractors, 150-message haystack, harness-validated 15/15 — changes nothing: 15/15, 2–5 calls. Episode traces expose the reason: the L3 answer is *pinned*, so *get_pinned_messages* provides an $O(1)$ retrieval path that no quantitative distractor count can defeat. **When knobs stop moving a model, the difficulty gap is structural.**

The response is the L4 family (Table 1): no pins; final figures embedded in plain history among superseded drafts (“early proposal — NOT final”); and a target quantity that exists nowhere in the workspace, forcing find → discriminate → aggregate → single threaded reply. Result: **40% pass — in band**. The three failures share an anatomy absent at every lower level: an opening *get_thread* call on a *hallucinated, descriptively-named post id* (“sofia_q3_figure_request”); prompt-sigil leakage (~channel passed verbatim to tools); then *exploration without commitment* — 15+ consecutive read/search calls, re-reading channels and re-issuing near-duplicate queries, ending at the 20-call budget with zero actions taken. The two passing episodes committed at 11 and 15 calls. The small model acts without verifying; the large model verifies without acting. Both pathologies are exactly what a binary, collateral-checked reward is positioned to train against, and the thin margin to the call budget identifies the budget itself as an additional difficulty knob.

7 Related Work

τ -bench [1] grades tool-using agents by comparing final database state against a reference — the grading model adopted here — and documents pass^k consistency failures we cite for frontier-scale failure modes our local models cannot exhibit. TheAgentCompany [2] evaluates agents on realistic workplace tasks (including chat) with checkpoint-based grading. WebArena [3] and AgentBench [4] provide fixed environment suites for web and multi-environment evaluation. All are *benchmarks*: fixed, hand-curated, built to measure. CHATOPS-GYM is a *generator* built to train, and its distinguishing machinery — consistency by construction, per-model calibration against a gradient band, and CI-validated graders with mutation testing — addresses needs benchmarks do not have. The mutation-testing idea transfers classical mutation analysis from test-suite quality [5] to reward-function quality.

8 Limitations and Future Work

Grading model. Final-state grading cannot verify ephemeral actions, pure-read tasks, prohibitions leaving no diff trace, or destroy-then-restore trajectories; audit-log grading would recover several of these. Typed matching is shallow NLP: it kills the cheap false-positive classes, not adversarial paraphrase. **Scale of evidence.** Reported cells use $n \leq 5$; the claims are about the methodology, and the pipeline re-runs at arbitrary n . **Coverage.** Four families over one application; planned metrics (API-surface coverage per family, prompt-embedding dispersion) would quantify curriculum diversity, and an LLM-assisted generation phase (paraphrase with semantic-preservation checks; template proposal gated by the same harness) is designed but not yet implemented. **Throughput.** Team-per-episode reset costs 15–20 s; a container pool with team namespacing is the designed path to rollout-scale concurrency.

References

- [1] S. Yao et al. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv:2406.12045*, 2024.
- [2] F. Xu et al. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. *arXiv:2412.14161*, 2024.
- [3] S. Zhou et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv:2307.13854*, 2023.
- [4] X. Liu et al. AgentBench: Evaluating LLMs as Agents. *arXiv:2308.03688*, 2023.
- [5] Y. Jia and M. Harman. An Analysis and Survey of the Development of Mutation Testing. *IEEE TSE*, 37(5), 2011.